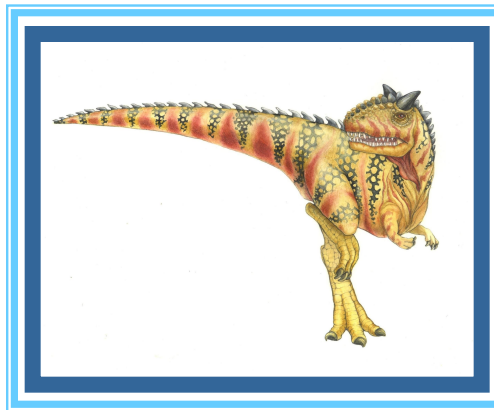


Unidad 3 Procesos





Unidad 3: Procesos

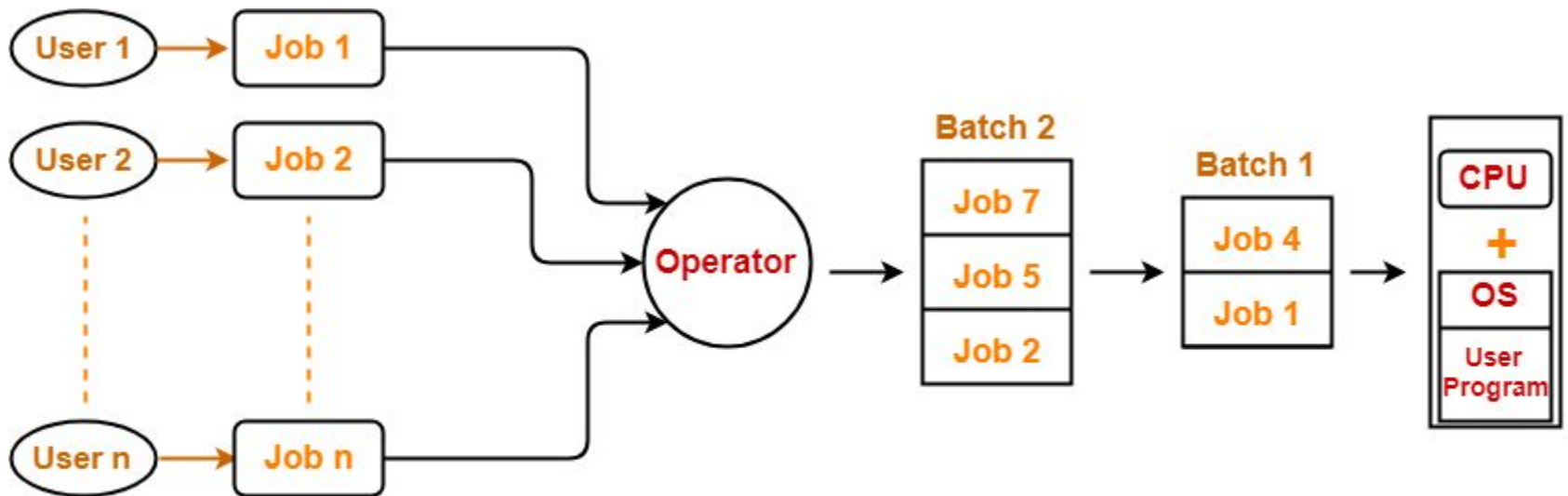
- El concepto de Proceso
- Planificación de Procesos
- IPC Interprocess Communication
- Ejemplos de IPC
- Comunicación en sistemas Client-Server





Sistema Batch

- Un SO ejecuta una variedad de programas:
 - Sistema Batch – **jobs** (batch = lote o grupo de tareas similares)



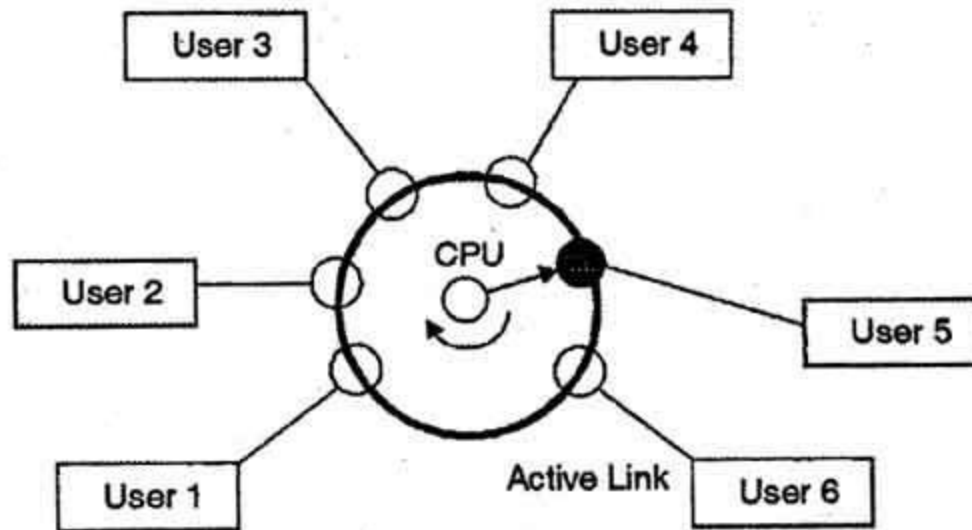
Batch Operating System





Sistema Time-Shared

- Un SO ejecuta una variedad de programas:
 - Sistema Time-shared – **programas de usuario** o **tareas**





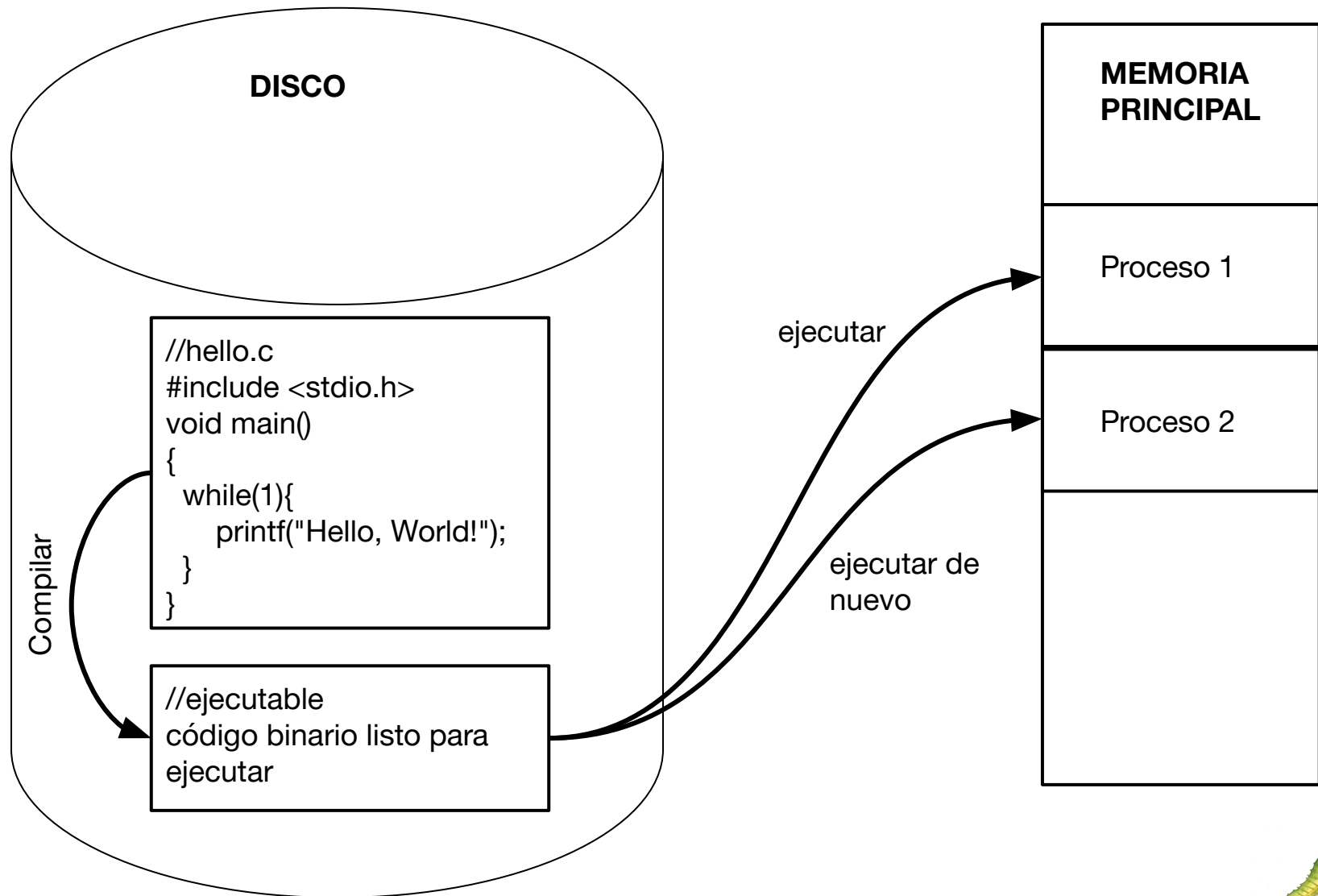
El concepto de Proceso

- **Proceso** – un programa en ejecución; La ejecución debe progresar de manera secuencial.
- Un programa es una entidad **pasiva** alojado en disco (**archivo ejecutable**), un proceso es una entidad **activa**
 - Un **programa** se convierte en **proceso** cuando es ejecutado y cargado en la memoria
- La ejecución de los programas puede ser vía GUI con un doble clic, a través de la línea de comandos , etc.
- Un programa puede ser varios procesos
 - Varias instancias de un mismo programa





El concepto de Proceso (Cont.)





El concepto de Proceso (Cont.)

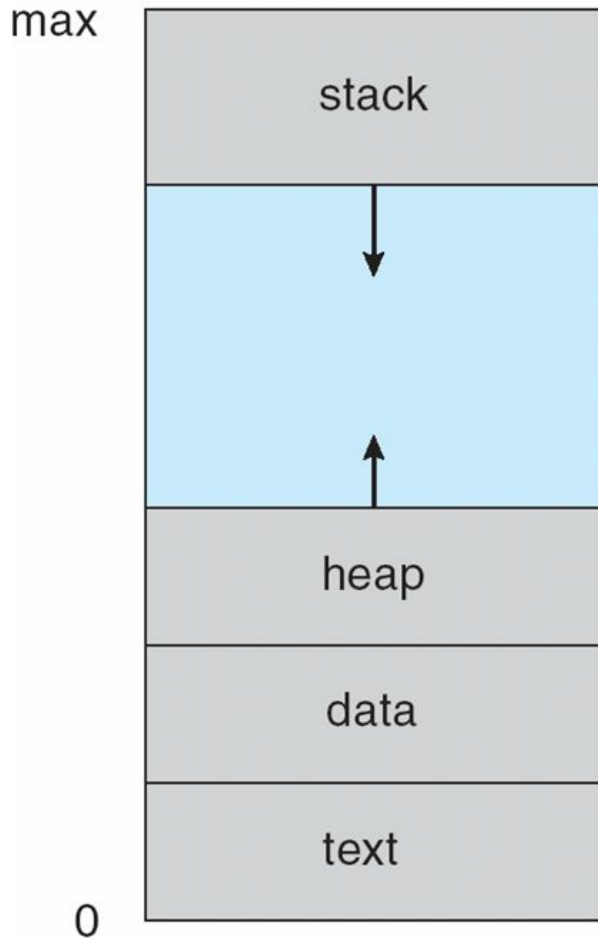
Ejercicio:

verificar que para un mismo programa podemos tener dos procesos asociados y diferentes





El interior de un proceso en la memoria



- Compuesto por:
 - El código del programa, también llamado **text section**
 - **Data section** contiene variables globales
 - **Heap** contiene memoria asignada dinámicamente durante el tiempo de ejecución
 - **Stack** que contiene datos temporales
 - Parámetros de funciones, variables locales, direcciones de retorno





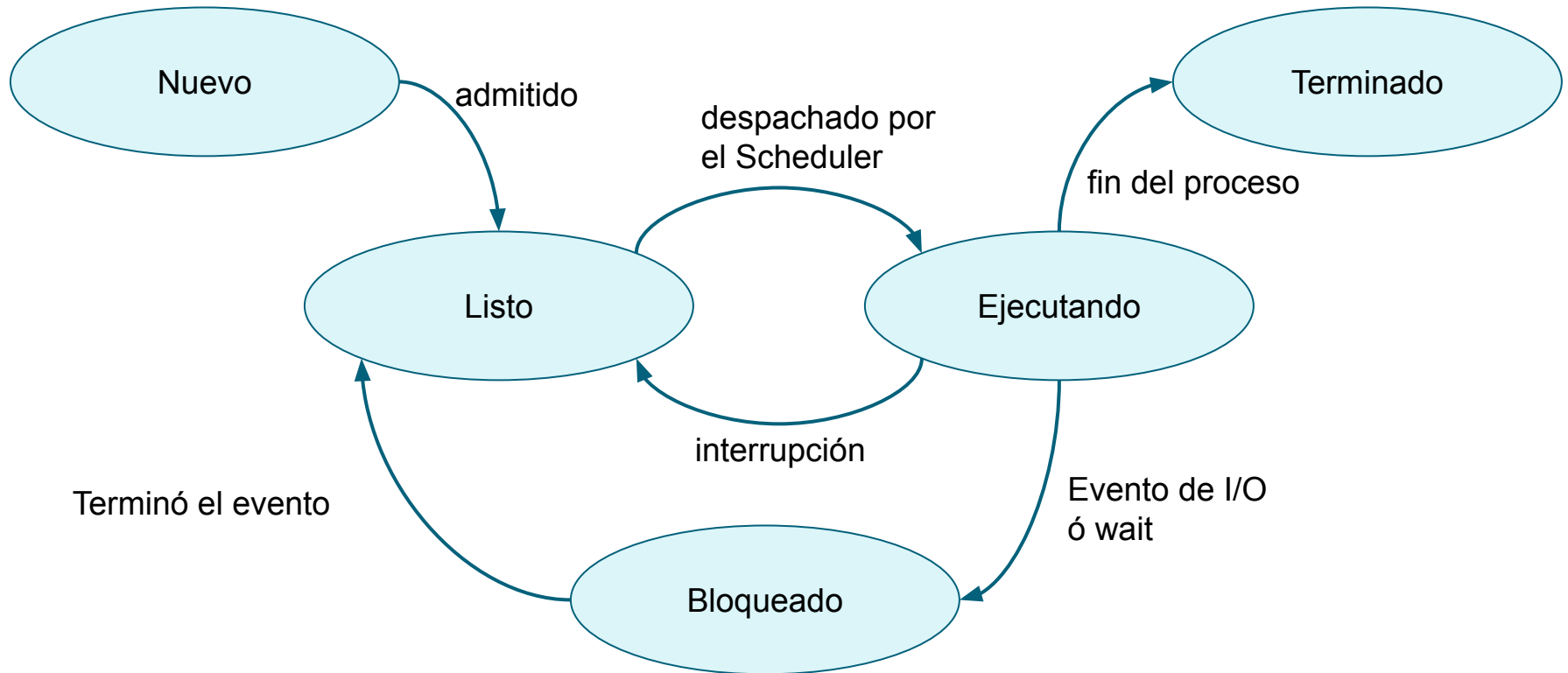
Estados de un Proceso

- A medida que se ejecuta un proceso, este cambia por distintos **estados**
 - **nuevo**: El proceso se está creando.
 - **ejecutando**: Las instrucciones están siendo ejecutadas
 - **esperando**: El proceso está esperando que ocurra algún evento.
 - **listo**: El proceso está a la espera de ser asignado a un procesador.
 - **terminado**: El proceso ha finalizado su ejecución.





Estados de un Proceso

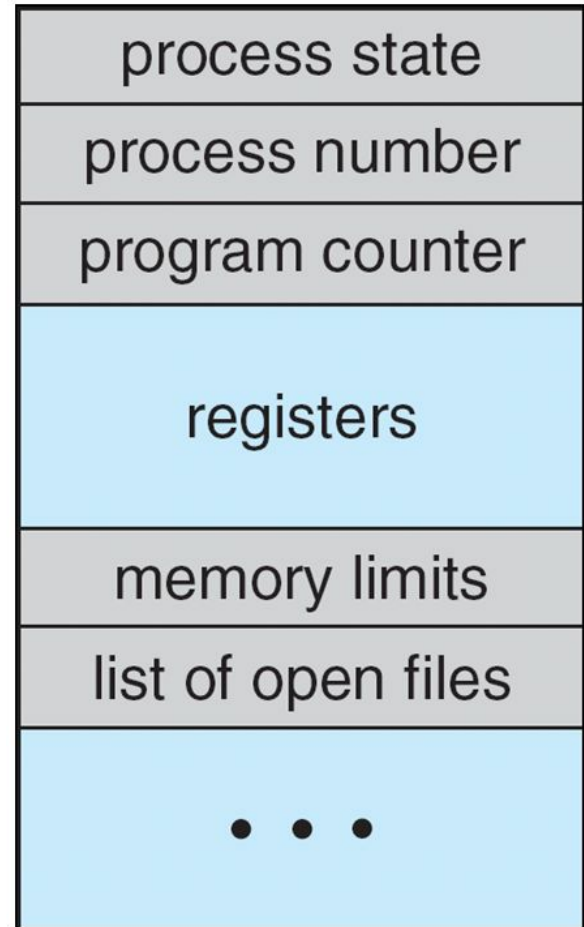




Process Control Block (PCB)

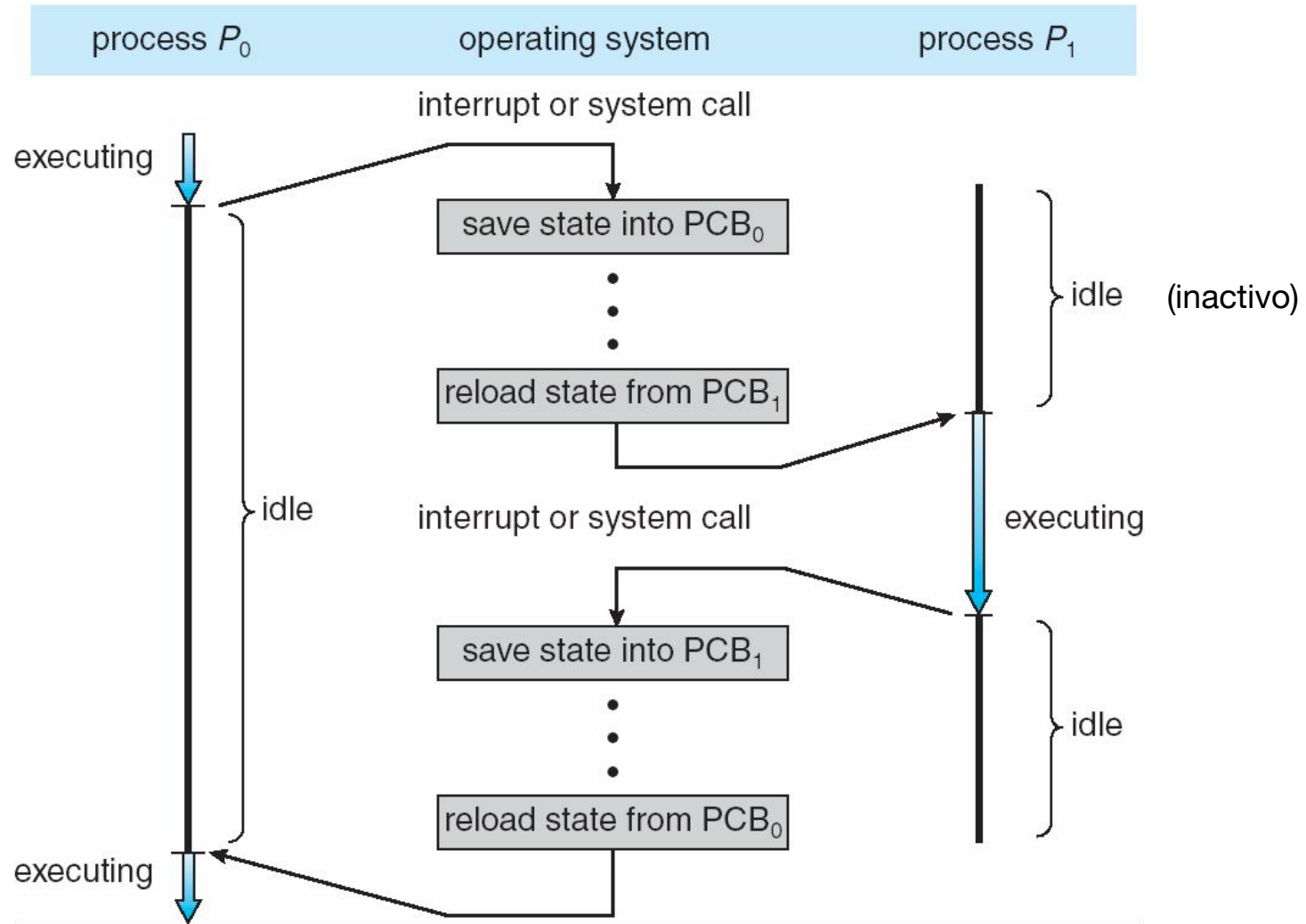
Bloque de Control del Proceso es la información asociada a cada proceso

- Process state – corriendo, esperando, etc
- Program counter – ubicación de la siguiente instrucción a ejecutar
- CPU registers – contenido de los registros del proceso
- CPU scheduling information- prioridad, scheduling queue pointers
- Memory-management información de la memoria asignada al proceso
- Información de ejecución – Uso de la CPU , clock transcurridos desde el inicio, tiempo límite
- Información de I/O – Dispositivos de E / S asignados a proceso, lista de archivos abiertos





PCB y Context Switch





Context Switch

- Cuando la CPU cambia de un proceso a otro, el sistema debe **guardar el estado** del proceso en ejecución y cargar el estado guardado del nuevo proceso, a través de un **context switch**
- **El Contexto** de un proceso es representado en el PCB
- El Context-switch es overhead; El sistema no hace ningún trabajo útil al cambiar los procesos
 - Mientras más complejo el SO y el PCB → más largo el context switch
- El tiempo es determinado por el hardware
 - Existe hardware que provee una colección de registros en la CPU que permite múltiples contextos simultáneos

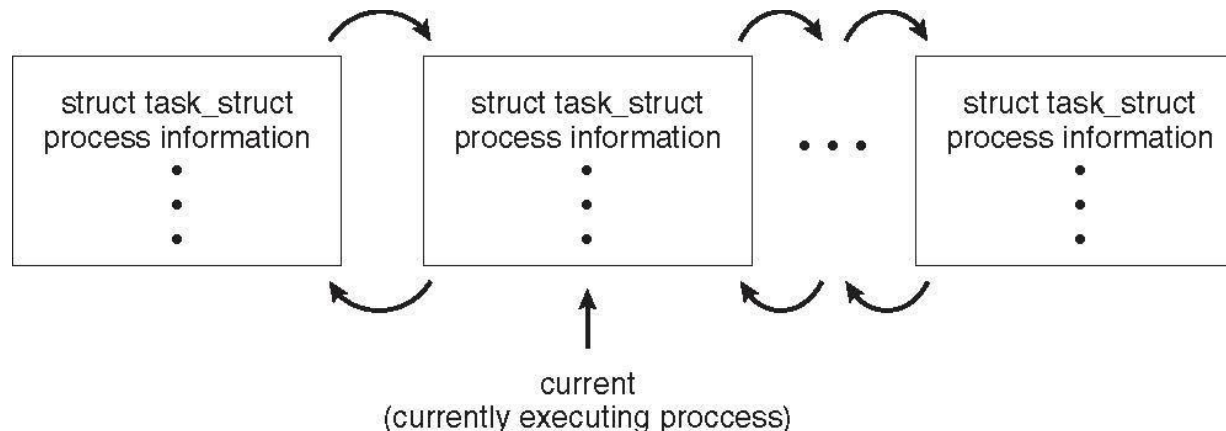




PCB: Representación en Linux

En Linux, la información de un proceso se guarda en un *PCB* llamado *task_struct*

```
pid t_pid; /* process identifier */
long state; /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm; /* address space of this process */
```





PCB: Representación en Linux

La estructura *task_struct* no está disponible directamente en user space, pero se puede acceder a cierta información del proceso con el siguiente comando:

```
cat /proc/<id_del_proceso>/status
```

Por ejemplo si el pid de un proceso es 2555

```
cat /proc/2255/status
```

nos muestra datos esenciales de proceso con id 2255

Cómo obtener el id de mi proceso (supongamos firefox)

```
top | grep "firefox"
```





Operaciones con Procesos

- El sistema debe proporcionar mecanismos para:
 - Creación de Procesos
 - Terminación de Procesos





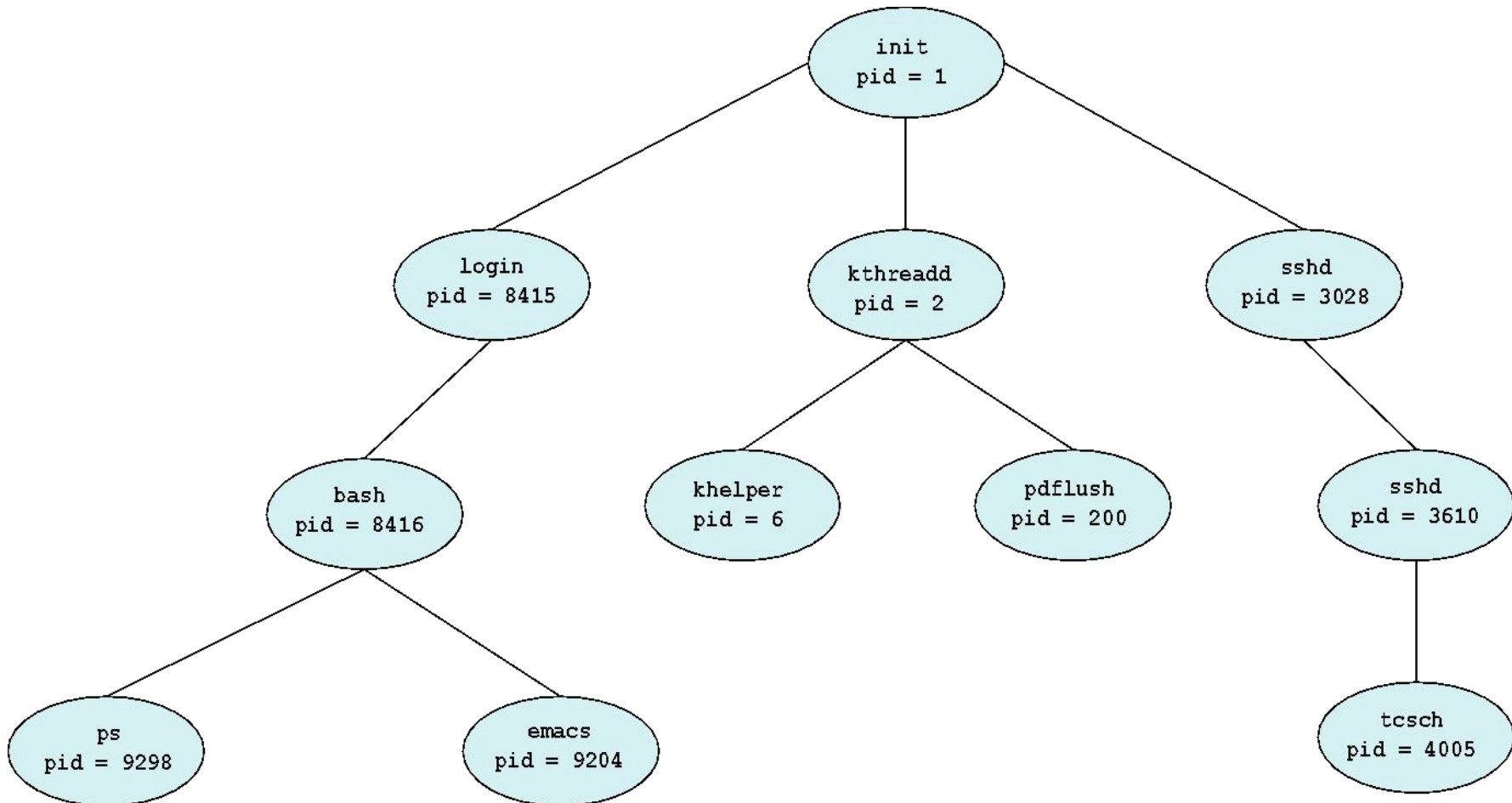
Creación de Procesos

- **Procesos Padres** pueden crear **Procesos hijos**, que a su vez crean otros procesos, formando un **árbol de procesos**.
- Los Procesos se identifican y gestionan a través de su **identificador de proceso (pid)**
- Opciones de intercambio de recursos
 - Padre e Hijo comparten todos los recursos
 - Los Hijos comparten un subconjunto de recursos del Padre
 - Padre e Hijo no comparten recursos
- Opciones de ejecución
 - Padre y Hijo se ejecutan **concurrentemente**.
 - El Padre espera por que el hijo finalice su ejecución





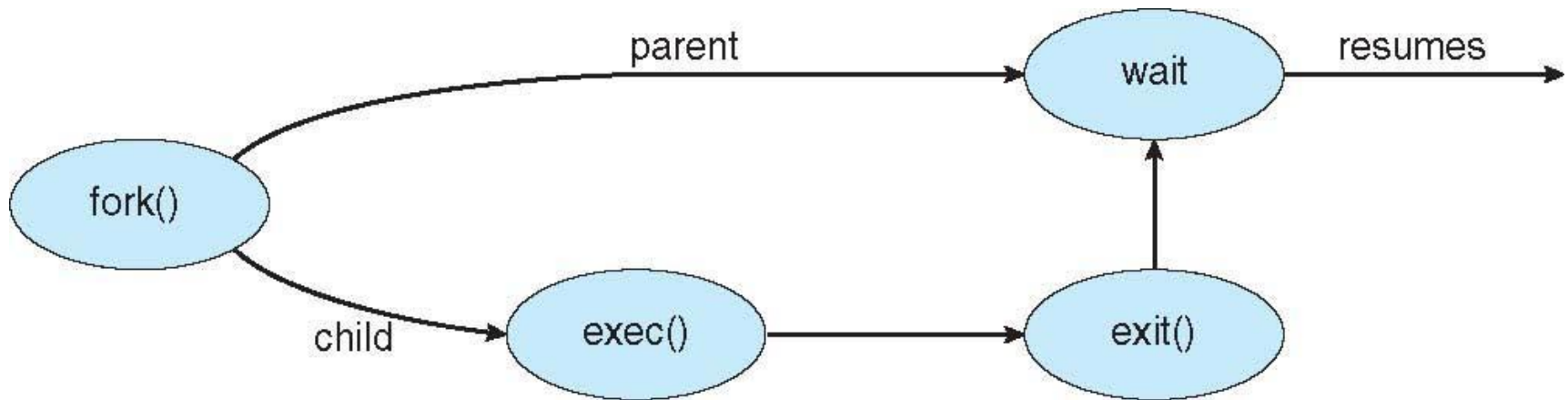
Árbol de procesos en GNU/Linux





Creación de Procesos (Cont.)

- Ejemplos UNIX
 - **fork()** system call que permite la creación de nuevos procesos
 - **exec()** system call que se usa luego de **fork()** para reemplazar el espacio de memoria del proceso con un nuevo programa
 - **wait()** system call para poner al padre en espera de que el hijo termine con un **exit()**





Programa C con Fork

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

    return 0;
}
```





Fin de un Proceso

- El proceso ejecuta su última instrucción y le solicita al sistema que lo elimine a través de la system call **exit()**
 - Retorna el estado del hijo al padre (via **wait()**)
 - Los recursos del proceso son reasignados por el SO.
- El padre puede terminar la ejecución de procesos hijos usando la system call **abort**. Algunas razones para esto:
 - El hijo supera los recursos asignados
 - La tarea asignada al hijo ya no es necesaria
 - El padre está siendo retirado y los sistemas operativos no permiten que un hijo continúe si su padre termina





Fin de un Proceso

- Algunos SO no permiten que exista un hijo si su padre ha terminado. Si un proceso termina, entonces todos sus hijos también deben ser terminados.
 - **Terminación en cascada.**
 - La terminación es iniciada por el SO
 - El proceso padre puede esperar la terminación de un proceso hijo usando la system call `wait()`. Esta system call devuelve información de estado y el pid del proceso terminado
- `pid = wait(&status);`
- Si ningún padre espera (no invoco `wait()`) el proceso es **zombie**
 - Si el padre terminó sin invocar `wait()`, el proceso **orphan**





Ejemplo Browser Chrome

- Algunos web browsers corren sobre un solo proceso (algunos todavía lo hacen)
 - Si un sitio web causa problemas, todo el navegador puede bloquearse
- Por otro lado, por ejemplo, Google Chrome es multiproceso, con 3 tipos diferentes de procesos:
 - **Browser** El proceso gestiona la interfaz de usuario, el disco y la red de E / S.
 - **Renderer** se encarga de hacer render en las páginas con HTML, Javascript. Un nuevo render es creado para cada sitio web abierto.
 - **Plug-in** Procesos para cada plug-in.





Interprocess Communication

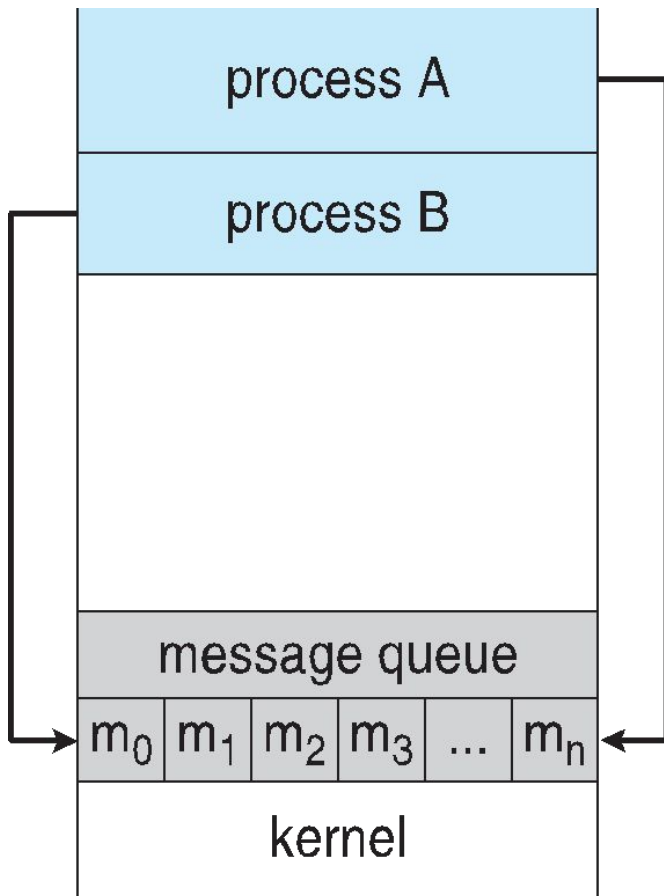
- Los procesos en un SO pueden ser ***independientes*** o ***cooperativos***
- El proceso de cooperación puede afectar o ser afectado por otros procesos, incluido el intercambio de datos
- Razones para que los procesos colaboren:
 - Intercambio de información
 - Mejora computacional
 - Modularidad
 - Conveniencia
- Los procesos cooperativos necesitan **interprocess communication (IPC)**
- Existen dos formas de IPC
 - **Shared memory (memoria compartida)**
 - **Message passing (paso de mensajes)**



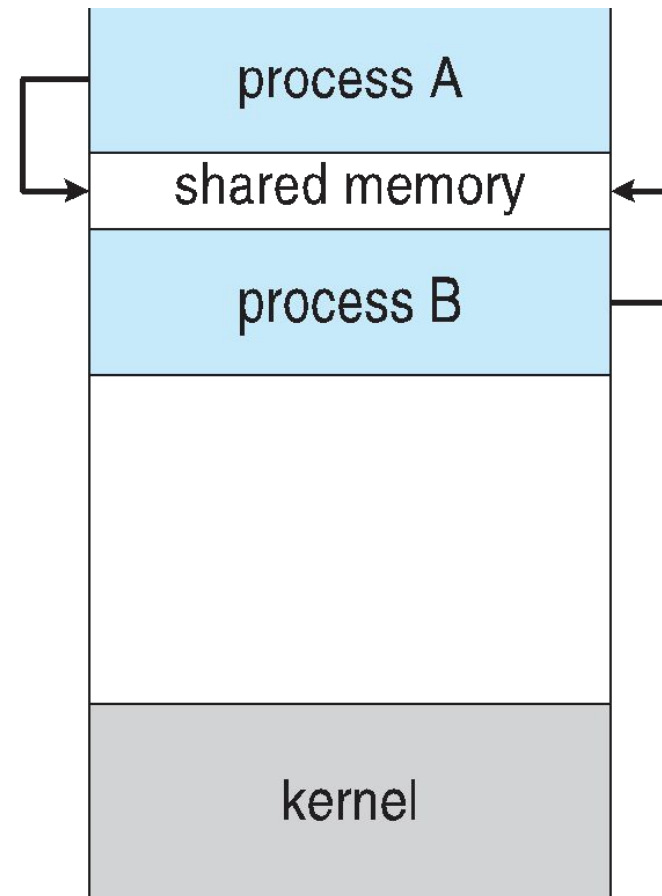


Modelos de Comunicación

(a) Envío de mensajes. (b) Memoria compartida.



(a)



(b)





IPC – Memoria Compartida

- Un área de memoria compartida entre los procesos que desean comunicarse.
- La comunicación está bajo el control de los procesos de los usuarios, no del sistema operativo.
- Los principales problemas son proporcionar un mecanismo que permita a los procesos del usuario sincronizar sus acciones cuando acceden a la memoria compartida.
- La **sincronización de procesos** se discute más adelante!





IPC – Paso de Mensajes

- Mecanismo para que los procesos se comuniquen y se sincronicen sus acciones.
- Sistema de mensajes: los procesos se comunican entre sí sin recurrir a variables compartidas
- IPC facilita esto, facilitando dos operaciones:
 - **send**(*message*)
 - **receive**(*message*)
- El tamaño del mensaje es fijo o variable.





IPC – Paso de Mensajes (Cont.)

- Si los procesos P y Q desean comunicarse , ellos necesitan :
 - Establecer un **communication link (enlace)** entre ellos
 - Intercambiar mensajes a través de send/receive
- Problemas para implementarlo:
 - ¿Cómo se establecen los enlaces?
 - ¿Se puede asociar un enlace a más de dos procesos?
 - ¿Cuántos enlaces puede haber entre cada par de procesos de comunicación?
 - ¿Cuál es la capacidad de un enlace?
 - En enlace es unidireccional o bidireccional?





IPC – Paso de Mensajes (Cont.)

- La implementación del enlace
 - Fisica:
 - Memoria compartida
 - Hardware bus
 - Red
 - Logica:
 - Directa o indirecta
 - Synchronous o asynchronous
 - Almacenamiento en búfer automático o explícito





Comunicación Directa

- Los procesos deben nombrarse explícitamente:
 - **send** (P , *message*) – envía un mensaje al proceso P
 - **receive**(Q , *message*) – recibe un mensaje del proceso Q
- Propiedades del enlace de comunicación.
 - Los enlaces se establecen automáticamente.
 - Un enlace está asociado exactamente con un par de procesos de comunicación.
 - Entre cada par existe exactamente un enlace.
 - El enlace puede ser unidireccional, pero generalmente es bidireccional.





Comunicación Indirecta

- Los mensajes son dirigidos y recibidos desde buzones (también conocidos como puertos/ports)
 - Cada PORT tiene una identificación única
 - Los procesos solo pueden comunicarse si comparten un PORT
- Propiedades del enlace
 - El enlace es establecido sólo si los procesos comparten puertos
 - Un link puede asociarse a varios procesos
 - Cada par de procesos puede compartir varios enlaces de comunicación.
 - Los Links pueden ser unidireccional o bidireccional





Comunicación Indirecta

- Operaciones
 - create a new mailbox (port)
 - send and receive messages through mailbox
 - destroy a mailbox
- Primitivas:
send(*A, message*) – envía un mensaje al puerto *A*
receive(*A, message*) –recibe un mensaje de el puerto *A*





Buffering

- Cola de mensajes asociada a un enlace
- implementada de una de tres maneras:
 1. Zero capacity – no hay mensajes en cola.
El remitente debe esperar al receptor
 2. Bounded capacity – longitud finita de n mensajes
El remitente debe esperar si el enlace está completo
 3. Unbounded capacity – longitud infinita
El remitente nunca espera



Fin de Unidad 2

